

Supplementary Requirements Documentation	2
Interactive House	2
Revision History	2
Color Code Legend	2
Supplementary Requirements List	2
Supplementary Requirements Descriptions	3
S1. Data persistency & consistency	3
S2. Database performance & latency	3
S3. Scalability of data storage	3
S4. Data security & access control	3
S5. Backup & recovery strategy	4
S6. Availability & fault tolerance	4
S7. Database technology constraints	4
S8. Deployment & Architecture Considerations	5

Supplementary Requirements Documentation

Interactive House

GitHub: [Interactive House](#) — see `SG1_Server_DB/` for server & database code

Revision History

Date	Version	Description	Author
06/02/2026	1.0	Initial supplementary requirements focusing on database POV	Katerina Arvay, Dario Ostojic, Andre Sandblom
01/03/2026	1.1	Update of supplementary requirements to reflect the adoption of Firebase Firestore Database	Katerina Arvay, Dario Ostojic, Andre Sandblom
17/03/2026	1.2	Extended supplementary requirements to include deployment considerations and hybrid architecture	Katerina Arvay, Dario Ostojic, Andre Sandblom
23/03/2026	1.3	Revised wording to include a dual database structure	Katerina Arvay, Dario Ostojic, Andre Sandblom
08/05/2026	1.4	S5 Not Implemented (Supabase is stub code); S4 Partial (SG1 Firestore rules wide open); S7 Partial (only primary DB operational); S8 elevated to Essential; priorities updated	Katerina Arvay, Dario Ostojic, Andre Sandblom

Color Code Legend

Essential — must be implemented for the system to function

Desirable — improves the system, but not strictly required

Supplementary Requirements List

Supplementary Requirement Name	Priority
S1. Data persistence & consistency	Essential
S2. Database performance & latency	Desirable
S3. Scalability of data storage	Desirable
S4. Data security & access control	Essential
S5. Backup & recovery strategy	Desirable

Supplementary Requirement Name	Priority
S6. Availability & fault tolerance	Desirable
S7. Database technology constraints	Desirable
S8. Deployment & Architecture Considerations	Essential

Supplementary Requirements Descriptions

S1. Data persistency & consistency

All smart home data, including user accounts, smart devices, device states, schedules, and historical logs, was persistently stored within the system's data storage solutions.

The system ensured logical data consistency such that stored device states accurately reflected the real-world state of devices after successful operations.

Where multiple updates occurred concurrently, appropriate mechanisms were used to prevent partial or inconsistent updates.

The system was designed to ensure consistency between primary and secondary data storage to avoid conflicting system states.

Real-time synchronization ensured that all connected clients received consistent and up-to-date device information.

S2. Database performance & latency

Database operations related to critical functionality, such as retrieving device status or updating device states, responded within a maximum of 2 seconds under normal operating conditions.

The system utilized Firebase's real-time data synchronization capabilities to ensure low-latency updates between clients and the database.

The database efficiently supported frequent read and write operations generated by multiple users and smart devices simultaneously.

S3. Scalability of data storage

The system was designed to support scalability to handle an increasing number of users, smart devices, automation rules, and sensor data over time.

The data storage solution was designed to support horizontal scaling and efficient handling of concurrent connections as the system grows.

S4. Data security & access control

The database enforced strict access control to ensure that users could only access data related to their own smart home environment.

Authentication and authorization mechanisms ensured that only authorized users and system components could access data.

Access control policies restricted read and write operations appropriately.

Sensitive information was not stored in plain text and followed secure handling practices.

Database access was limited to authorized backend components only. Note: the SG1 Firebase project's Firestore security rules are currently a temporary development setting allowing all reads and writes until October 2026. Proper role-based rules must be deployed before production use.

S5. Backup & recovery strategy

The system was intended to implement a backup and recovery strategy using a secondary data storage solution. This was not implemented.

Critical system data was intended to be replicated or backed up to ensure durability in case of failure of the primary database. This was not implemented.

The system was intended to support recovery to a recent consistent state in the event of data loss, corruption, or system failure. This was not implemented.

Backup mechanisms were intended to minimize data loss and ensure continuity of system operation. This requirement was not implemented. The Supabase folder contains only placeholder stub code. The system relied entirely on Firebase Firestore with no backup or recovery path. This remains as future work.

S6. Availability & fault tolerance

The system relied on Firebase's distributed cloud infrastructure to ensure high availability.

Firebase's built-in replication and redundancy mechanisms minimized downtime and reduced the risk of data loss due to hardware or software failures.

The system continued operating under partial network disruptions, with automatic synchronization when connectivity was restored.

The system was designed to continue operating in a degraded mode if one data storage solution became unavailable, where possible.

S7. Database technology constraints

The system used a primary NoSQL database (Firebase Firestore). The secondary relational database (Supabase) was not implemented.

The primary database supported real-time synchronization of device states between clients and the backend.

The secondary database was intended to provide structured storage and support backup and recovery functionality. It was not implemented.

The data model was designed to accommodate both NoSQL and relational storage paradigms. Only the primary NoSQL database (Firebase Firestore) was operational. The secondary relational database (Supabase) was not implemented.

S8. Deployment & Architecture Considerations

The system is deployed using a hybrid architecture. The web application is deployed as a cloud-hosted web service (Render), while the backend communication layer is handled through a locally hosted gateway service.

The gateway exposed a REST API and was responsible for forwarding commands to the smart house hardware and updating the resulting state in Firebase Firestore.

To enable external access to the locally hosted gateway, Cloudflare Tunnel was used to provide a publicly accessible API endpoint without requiring full backend deployment to the cloud.

This approach reduced backend complexity while preserving real-time synchronization and ensuring that device state reflected actual hardware behavior.

The architecture was designed to support integration with both primary and secondary data storage solutions to ensure reliability and data durability.